

Транзакция на метод бизнес-логики

Транзакция — это группа последовательных операций, которая представляет собой логическую единицу работы с данными. Она может быть выполнена целиком и успешно, соблюдая целостность данных, и независимо от параллельно идущих других транзакций. В противном случае транзакция не выполнена вообще, и тогда она не должна произвести никакого эффекта, а если в процессе выполнения транзакции над данными были произведены какие-либо изменения, то они будут возвращены к состоянию до начала транзакции.

В контексте [методов БЛ](#), когда внутри методов инициируется вызов других методов, транзакции определяют поведение методов в зависимости от важности результатов их выполнения. Например, этот функционал используется в операциях биллинга.

При запросе к БД из исходного кода по умолчанию создаётся транзакция (см. [транзакции PostgreSQL](#)). Однако разработчик может самостоятельно устанавливать место создания транзакции в исходном коде с помощью методов [CreateTransaction](#) в C++ или [CreateTransaction](#) в Python. Все транзакции на Wasaby Framework создаются по типу "ленивых", подробнее о которых вы можете прочитать [здесь](#).

Внутри транзакции допускается вызов любых методов бизнес-логики, которые производят запрос к БД. Вызов методов можно осуществлять, например, в Python с помощью метода [InvokeBLMethod](#). Вызываемый метод будет вовлечён в существующую транзакцию. Если производится вызов метода с произвольным количеством параметров ([Custom method](#)), то для него допускается [конфигурация типа транзакции](#), в которую он будет вовлечён.

Вызовы методов выполняются внутри одного и того же сервиса.

В приложении вызов метода БЛ может быть инициирован пользователями с разными правами доступа. Чтобы управлять логикой SQL запросов в зависимости от прав доступа, можно получить список идентификаторов пользователей текущей области видимости. Идентификаторы доступны в т.н. переменной транзакции, подробнее о которой вы можете прочитать [здесь](#).

При работе с транзакциями на Python существуют ограничения, подробнее о которых вы можете прочитать [здесь](#). Также обратите внимание на [общие рекомендации](#) при работе с БД в транзакционном режиме pgBouncer.

Механизм ленивых транзакций

В Wasaby Framework реализован механизм создания т.н. "ленивых" транзакций. Объект транзакции может быть создан (см. метод [CreateTransaction](#)) в любом месте исходного кода. Он инициирует создание "клиентского" соединения с менеджером pgBouncer. Однако "серверное" соединение будет создано только в момент запроса к БД. Такое поведение может сэкономить ресурсы сервера БД, и часто эффективно для "небольших" транзакций.

Пример (выдержка из прикладного кода). В следующем примере транзакция (и соединение с БД) будет создана только внутри функции "temporary_documents", если перед этим не происходит исключения (генерируется по невыполнению условия).

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
RPC_FUNC_1(
L"ФормализованныйДокумент.СписокФайлов",
bl::MethodListResult, GetTemporaryDocumentsByGuid,
bl::MethodListParams, L"ПараметрыСписка", params )
{
    // Создаём экземпляр класса для работы с БД.
    sbis::dbi::IDatabase* db =
sbis::dbi::GetConnectedDataDB();

    // Создаём транзакцию (ленивая транзакция).
Соединение с БД пока не производится.

    sbis::dbi::Transaction tr = db-
>CreateTransaction(
sbis::dbi::Transaction::READ_COMMITTED,
sbis::dbi::Transaction::WRITE );

    // Выполняем проверку: нужен обязательный
параметр.

    if( params.filter->TestField( L"Ид" ) ==
nullptr || (*params.filter)[L"Ид"].To<String>() ==
L" " )
    {
        /* Если условие не выполняется, то создаём
исключение. Объект транзакции уничтожается.
```

Соединение с БД не производится.

За счет того, что не было создано соединение, которое не использовалось для запросов, и производится экономия ресурсов. */

```
Error<ImplException<class RPCConvertType>>(
String( L"Не задан обязательный входной параметр
\Ид\" (GUID)")) );
}
```

```
String guid_string = (*params.filter)[
L"Ид"].To<String>();
```

/* Производится вызов функции, внутри которой осуществляется запрос к БД.

Здесь создаётся транзакция и, соответственно, соединение с БД. */

```
TemporaryDocuments temporary_documents (
guid_string );
```

```
result.cursor =
temporary_documents.ListDocuments ( *db )-
>Cursor();
```

```
} // Здесь (scope) транзакция будет уничтожена, и
закрывается соединение с БД.
```

Для прикладного разработчика не требуется производить какую-либо дополнительную настройку. Все транзакции создаются по типу "ленивых". Функционал реализован в библиотеке "sbis-dbi" и поставляется совместно с [SBIS SDK](#).

Конфигурация типа транзакции в Genie

Конфигурация типа транзакций допустима только в методах с произвольным количеством параметров ([Custom method](#)). С её помощью устанавливают поведение и уровень изоляции данных при вызове метода из исходного кода или внутри других методов бизнес-логики. За конфигурацию [типа транзакции](#) отвечает опция "Transaction type":

Рис. 1. Опция "Transaction type" в настройках метода бизнес-логики в Genie.

Такое декларативное описание транзакции упрощает исходный код. При этом не возникает необходимости выбора уровня изоляции транзакции, который всегда один — [Read Committed](#). Кроме того другим разработчикам по описанию метода сразу видно, как он работает с транзакциями. Не нужно смотреть реализацию метода, что в случае встроенных функций весьма проблематично.

В настройках метода отсутствует опция Transaction type

Если в настройках метода бизнес-логики отсутствует опция "Transaction type", тогда такой метод вовлекается в существующую транзакцию, а новая — не создается. Аналогично типу транзакции [Supported](#).

Типы транзакций

Поддерживается (Supported). По умолчанию

Метод вовлекается в существующую транзакцию, а новая — не создается. Является поведением по умолчанию для всех методов бизнес-логики. Тип транзакции для метода "А" в примерах ниже - "Поддерживается" (Supported).

- Если метод "А" вызывается из метода "В" с типом транзакции "Не поддерживается", то он будет выполнен тоже без транзакции.
- Если метод "А" вызывается из метода "В" с типом транзакции "Требуется" или "Требуется новая", то он будет выполнен в транзакции метода "В". Это означает, что изменения данных, произведённые в методе "В", будут доступны внутри метода "А". В случае SQL ошибок будет прервана исходная транзакция.
- Если метод "А" выполнится с ошибкой, и он был включен в существующую транзакцию, то все изменения, созданные им и другими методами той же транзакции, будут возвращены к начальному состоянию.

Не поддерживается (Unsupported)

Метод гарантированно выполняется без транзакции. Тип транзакции для метода "А" в примерах ниже - "Не поддерживается" (Unsupported).

- Если метод "А" будет вызван из метода "В", то изменения данных, произведённые в методе "В", не будут доступны в методе "А".
- Если метод "А" выполнится с ошибкой, то все созданные им изменения данных не будут возвращены к начальному состоянию. Транзакция внешнего метода "В", из которого был вызван метод "А", останется без изменений данных.

Поведение методов с транзакцией типа "Не поддерживается" обеспечивается созданием отдельного соединения с БД. Поэтому выбор этого типа транзакции должен быть серьёзно обоснован.

Требуется (Required)

Метод будет вовлечен в существующую транзакцию. Если её нет, для метода будет создана новая транзакция. Тип транзакции для метода "А" в примерах ниже - "Требуется" (Required).

- Если метод "А" выполняется в новой транзакции, то в случае ошибки все созданные им изменения будут возвращены к начальному состоянию.
- Если метод "А" вовлекается в существующую транзакцию метода "В", то в случае ошибки возврату к начальному состоянию подлежат изменения как самого метода, так и других методов той же транзакции.

Требуется новая (Required new)

Метод гарантированно выполняется в отдельной транзакции. Тип транзакции для метода "А" в примерах ниже - "Требуется новая" (Required new).

- Если метод "А" вызывается из метода "В" с типом транзакции "Требуется" или "Требуется новая", то изменения данных, произведённые в методе "В", не будут доступны в методе "А".
- Если метод "А" вызывается внутри "try catch" и он выполнится с ошибкой, то все созданные им изменения будут возвращены к начальному состоянию. Транзакция внешнего метода "В" останется без изменений.
- Если метод "А" выполнится с ошибкой, то она будет передана во внешний метод "В", который завершается с ошибкой и откатывает свою транзакцию.

Поведение методов с транзакцией типа "Требуется новая" обеспечивается созданием отдельного соединения с БД. Поэтому выбор этого типа транзакции должен быть серьёзно обоснован.

Точка сохранения

При работе внутри транзакции можно создать именованную точку сохранения. Это делается с помощью методов `CreateSavePoint` и `ReleaseSavePoint` у класса [Transaction](#).

[Savepoint](#) - это именованная точка, к которой впоследствии можно откатиться, т.е. можно откатить не всю транзакцию, а только до Savepoint.

В рамках одной транзакции могут существовать несколько точек сохранения. Savepoint полезны для реализации комплексного восстановления ошибок в приложениях баз данных. Если во время транзакции с несколькими операциями возникает ошибка, приложение может восстановиться после ошибки (путем возврата к точке сохранения) без необходимости прерывать всю транзакцию.

Savepoint создается через экземпляр класса `Transaction`:

- [C++](#):

1	<code>auto tr = GetConnectedDataDB()-</code>
2	<code>>CreateTransaction();</code>
3	<code>...</code>
4	<code>tr.CreateSavePoint(L"savepoint1");</code>
	<code>...</code>

- [Python](#):

1	<code>with sbis.CreateTransaction(...) as txn:</code>
2	<code>...</code>
3	<code>txn.CreateSavePoint("savepoint1")</code>
4	<code>...</code>

Откат к SavePoint выполняется с помощью функции `RollbackToSavePoint`:

- [C++](#):

1	<code>auto tr = GetConnectedDataDB()-</code>
2	<code>>CreateTransaction();</code>
3	<code>...</code>
4	<code>tr.CreateSavePoint(L"savepoint1");</code>
5	<code>...</code>
6	<code>tr.RollbackToSavePoint(L"savepoint1");</code>
	<code>...</code>

- [Python:](#)

1	<code>with sbis.CreateTransaction(...) as txn:</code>
2	<code>...</code>
3	<code>txn.CreateSavePoint("savepoint1")</code>
4	<code>...</code>
5	<code>txn.RollbackToSavePoint("savepoint1")</code>
6	<code>...</code>

Если некоторый фрагмент бизнес-логики был выполнен успешно, то существующую точку восстановления можно удалить с помощью метода `ReleaseSavePoint`:

- [C++:](#)

1	<code>auto tr = GetConnectedDataDB()-</code>
2	<code>>CreateTransaction();</code>
3	<code>...</code>
4	<code>tr.CreateSavePoint(L"savepoint1");</code>
5	<code>...</code>
6	<code>tr.ReleaseSavePoint(L"savepoint1");</code>
	<code>...</code>

- [Python:](#)

1	<code>with sbis.CreateTransaction(...) as txn:</code>
2	<code>...</code>
3	<code>txn.CreateSavePoint("savepoint1")</code>
4	<code>...</code>
5	<code>txn.ReleaseSavePoint("savepoint1")</code>
6	<code>...</code>

Откат к `SavePoint` сбрасывает флаг отката транзакции по исключению.

Работа с транзакциями на Python

В Python в ряде неочевидных случаев возможны [утечки памяти](#). При работе с транзакциями утечка приводила не только к тому, что росла память, занимаемая воркером (web-worker) — транзакция не закрывалась, и все последующие вызовы попадали в неё же. Через некоторое время она откатывалась, что приводило к потере изменений, которые успели сделать успешно завершённые методы, попавшие в неё.

Поэтому вводится обязательное требование к использованию блока **with**.

Если вы явно создаете транзакцию, например:

1	<code>tr = CreateTransaction(</code>
2	<code>TransactionLevel.READ_COMMITTED,</code>
3	<code>TransactionMode.WRITE)</code>
4	<code>SqlQuery('INSERT INTO "SomeTable1" DEFAULT VALUES'</code>
	<code>)</code>
	<code>SqlQuery('INSERT INTO "SomeTable2" DEFAULT VALUES'</code>
	<code>)</code>
	<code>return True</code>

то правильно будет записать это так:

1	<code>with CreateTransaction(</code>
2	<code>TransactionLevel.READ_COMMITTED,</code>
3	<code>TransactionMode.WRITE):</code>
4	<code>SqlQuery('INSERT INTO "SomeTable1" DEFAULT</code>
	<code>VALUES')</code>
	<code>SqlQuery('INSERT INTO "SomeTable2" DEFAULT</code>
	<code>VALUES')</code>
	<code>return True</code>

Использование [CreateTransaction](#) без блока **with** считается устаревшим. В логи будет записываться соответствующая ошибка.

При прерывании вложенной транзакции, по сути, внешняя транзакция помечается на откат (вложение транзакций лишь логическое, с точки зрения БД транзакция одна вне зависимости от глубины вложенности транзакций на БЛ).

Внешнюю транзакцию можно откатить к точке сохранения после прерывания вложенной, если точка сохранения была создана до открытия вложенной транзакции. После отката работа с внешней транзакцией может быть продолжена.

Общие рекомендации при работе с БД в транзакционном режиме pgBouncer

pgBouncer — это мультиплексор соединений (см. [PgBouncer — PostgreSQL wiki](#)), который в случае приложений СБИС работает в транзакционном режиме.

Если в прикладном коде разработчик не производит открытие транзакции перед запросом к БД, то каждая новая инструкция, возможно, будет исполняться через pgBouncer в новом серверном соединении с БД. Однако возможны ситуации, при которых в новом соединении с БД переменные сессии будут отсутствовать, или их значения будут иными. Поэтому если программист явно не открывает транзакцию, то механизм платформы производит это неявно на каждую инструкцию, далее устанавливает набор переменных сессии, производит выполнение инструкции и в конце либо закрывает транзакцию (commit), либо откатывает изменения (rollback). Такая процедура создаёт дополнительную нагрузку на сервер БД. Чтобы избежать повышения нагрузки, можно:

1. **Создать транзакцию явно.** Т.е. в исходном коде с помощью метода `CreateTransaction` в [C++](#) или в [Python](#) обозначать начало транзакции. Её закрытие будет производиться автоматически при изменении области видимости.
 - Если производится подряд несколько запросов к БД, например, через `Select` на выборку данных, то их нужно обернуть в одну отдельную транзакцию. Это позволит работать с согласованными между собой данными с учетом уровня изоляции транзакции. При этом нужно учитывать, что "длинные" транзакции вредны для сервера БД.
 - Если в транзакции не будут производиться изменения данных, то её нужно создавать в режиме "READ" (параметр `mode` для методов `CreateTransaction`).
2. **Установить конфигурацию транзакции для метода в Genie.** Эта настройка будет учтена при вызове метода из исходного кода. Подробнее об этом вы можете прочитать [здесь](#).

Не создавайте "длинных" транзакций. Если логика внутри транзакции такова "Запрос к БД → Запрос на сторонний сервис → Обработка данных с сервиса → Запрос к БД", то её нужно изменить. Запрос на сервис лучше исключить из транзакции, а первый и последний запрос к БД выполнять в отдельных транзакциях. Баланс между "короткими" и "длинными" транзакциями разработчик должен определить самостоятельно в зависимости от ситуации.

Обработчики на завершение транзакций

Встречаются ситуации, требующие использовать обработчики на завершение транзакций. Рассмотрим пару подобных сценариев.

Пример **негативного сценария**.

Одно из действий блока транзакции отработало с определенным результатом. Например, был сформирован файл с отчетом об операциях. Однако транзакция не была выполнена, прошел откат изменений, и, соответственно, файл необходимо удалить. То есть, необходим обработчик на `rollback`, который удалит ненужный файл.

Пример **позитивного сценария**.

Представим банковскую транзакцию, результатом которой будет передача денежных средств от абонента *Васи* абоненту *Пете*. Логичным завершением операции перевода будет оповещение абонентов. *Васю* — о том, что его счет уменьшился на ожидаемую сумму, *Петю* — о том, что его счет был пополнен. Однако мы не можем оповестить абонентов, не зная, как завершится транзакция. В этой ситуации нам нужен обработчик на `commit`, который отправит необходимые сообщения в случае успешного завершения транзакции.

Неименованные обработчики

Неименованный `callback` представляет из себя [функтор](#).

Один и тот же функтор может быть установлен неограниченное количество раз в рамках одной транзакции и может быть одновременно установлен как на фиксацию, так и на откат транзакции. Функтор будет вызван столько раз, сколько был установлен на наступивший результат транзакции.

Неименованный `callback` может быть добавлен с помощью глобальных методов (только в Python):

- [InvokeOnTransactionCommit\(callable \)](#) - на фиксацию транзакции;
- [InvokeOnTransactionRollback\(callable \)](#) - на откат транзакции.

2	<code>import event</code>
3	<code>...</code>
4	<code>def on_commit(users, rid):</code>
5	<code> mess = Record()</code>
6	<code> mess.AddArrayInt64('Users').From(users)</code>
7	<code> mess.AddArrayString('DeleteRoles').From(</code>
8	<code>[rid])</code>
9	<code> event.LocalPublish('rights.changed.local.roles'</code>
10	<code>, mess)</code>
	<code> ...</code>
	<code>sbis.InvokeOnTransactionCommit(partial(on_commit,</code>
	<code>real_users, RoleId))</code>

Или через методы экземпляра класса [sbis.IDatabase](#) в Python:

- [InvokeOnTransactionCommit\(callable \)](#) - на фиксацию транзакции;
- [InvokeOnTransactionRollback\(callable \)](#) - на откат транзакции.

И [sbis::dbi::IDatabase](#) в C++:

- [AddCallbackOnCommit\(std::function<void\(\)> callback \)](#) - на фиксацию транзакции;
- [AddCallbackOnRollback\(std::function<void\(\)> callback \)](#) - на откат транзакции.

1	<code>sbis::dbi::GetConnectedDataDB()-</code>
2	<code>>AddCallbackOnRollback([report]() {</code>
3	<code> DeleteReport(report);</code>
	<code>});</code>

Именованные обработчики

Именованный `callback` - в C++ это объект, реализующий интерфейс [sbis::INamedCallback](#) и имеющий непустую строку имени, в Python это объект, имеющий строковый атрибут `name` с непустым значением и метод `execute()` для обратного вызова.

В отличие от *неименованных* обработчиков *именованные* могут быть установлены лишь единожды в рамках одной транзакции, и вызываются не более одного раза при её завершении. Это позволяет предупреждать множественные установки обработчиков общих компонент (попытка добавить обработчик с именем, идентичным одному из уже добавленных, закончится выбросом исключения).

Например, такой обработчик может использоваться для выполнения однотипных операций (удаления временных файлов). В этом случае при добавлении аналогичной операции, сначала следует получить обработчик по названию и, только если таковой не найден, добавить новый.

Интерфейс *именованных* обработчиков класса [sbis.IDatabase](#) в Python:

- [AddCallbackOnCommit\(callback \)](#) - для установки обработчика на фиксацию транзакции;
- [AddCallbackOnRollback\(callback \)](#) - для установки обработчика на откат транзакции;
- [GetCallbackOnCommit\(name \)](#) - для поиска обработчика на фиксацию транзакции по имени;
- [GetCallbackOnRollback\(name \)](#) - для поиска обработчика на откат транзакции по имени.

1	<code>class SalaryDocRunner:</code>
2	<code> def __init__(self, doc_id):</code>
3	<code> self.name = 'SalaryDocRunner_' + doc_id</code>
4	<code> self.__doc_id = doc_id</code>
5	<code> def execute(self):</code>
6	<code> Документооборот.StartDoc(self.__doc_id)</code>
7	<code> ...</code>
8	<code> if not</code>
9	<code>sbis.GetConnectedDatabase().GetCallbackOnCommit(' Sa</code>
	<code>laryDocRunner_' + doc_id):</code>
	<code> sbis.GetConnectedDatabase().AddCallbackOnCommit</code>
	<code>(SalaryDocRunner(doc_id))</code>

Интерфейс *именованных* обработчиков класса [sbis::dbi::IDatabase](#) в C++:

- [`AddCallbackOnCommit\( std::unique\_ptr<INamedCallback> callback \)`](#) - для установки обработчика на фиксацию транзакции;
- [`AddCallbackOnRollback\( std::unique\_ptr<INamedCallback> callback \)`](#) - для установки обработчика на откат транзакции;
- [`GetCallbackOnCommit\( StringView name \)`](#) - для поиска обработчика на фиксацию транзакции по имени;
- [`GetCallbackOnRollback\( StringView name \)`](#) - для поиска обработчика на откат транзакции по имени.

1	<code>class FunctionTransactionCallback : public</code>
2	<code>sbis::INamedCallback</code>
3	<code>{</code>
4	<code>public:</code>
5	<code>FunctionTransactionCallback(sbis::StringView</code>
6	<code>name, std::function< void() > func)</code>
7	<code>: mFunc{ std::move(func) }</code>
8	<code>{</code>
9	<code>SetName(name);</code>
10	<code>}</code>
11	<code>void Execute() override</code>
12	<code>{</code>
13	<code>mFunc();</code>
14	<code>}</code>
15	<code>private:</code>
16	<code>std::function< void() > mFunc;</code>
17	<code>};</code>
18	<code>...</code>
19	<code>constexpr auto callback_name =</code>
20	<code>L"EmitRefreshEventPayloadOnRollback"_sv;</code>
21	<code>if(!sbis::dbi::GetConnectedDataDB()-</code>
22	<code>>GetCallbackOnRollback(call_back_name))</code>
23	<code>{</code>
	<code>sbis::dbi::GetConnectedDataDB()-</code>
	<code>>AddCallbackOnRollback(</code>
	<code>std::make_unique<</code>
	<code>FunctionTransactionCallback >(</code>
	<code>callback_name, [this]() {</code>
	<code>EmitRefreshEventPayloadOnRollback(); }));</code>
	<code>}</code>

Политика исполнения функций обратного вызова

На любые исключения, отданные обработчиком (не важно — `commit` или `rollback`) будет залогировано сообщение об ошибке. Все следующие обработчики будут выполнены.

При отсутствии на соединении открытой транзакции во время установки обработчика коллбэки на `commit` исполняются немедленно, коллбэки на `rollback` отбрасываются.

Политика обработки ошибок

В этом разделе описываются сценарии поведения транзакций при прерывании.

Завершение внешней транзакции влечёт выброс исключения, если вложенная транзакция была прервана, а внешняя - нет

1	<code>try:</code>
2	<code>with sbis.CreateTransaction(...) as outer:</code>
3	<code>try:</code>
4	<code>with sbis.CreateTransaction(...) as</code>
5	<code>middle:</code>
	<code>try:</code>

6	<code>with</code>
7	<code>sbis.CreateTransaction(...) as inner:</code>
8	<code>inner.Break()</code>
9	<code>except:</code>
10	<code># Сюда не попадаем, т.к. мы</code>
11	<code>явно прервали текущую (inner) транзакцию.</code>
12	<code>pass</code>
13	<code>except:</code>
14	<code># Попадаем сюда, т.к. вложенная</code>
15	<code>транзакция прервана, а текущую (middle) мы явно не</code>
16	<code>прерывали.</code>
17	<code>pass</code>
	<code>outer.Break()</code>
	<code>except:</code>
	<code># Сюда не попадаем, т.к., хотя вложенная</code>
	<code>транзакция прервана, мы явно прервали текущую</code>
	<code>(outer) транзакцию.</code>
	<code>pass</code>

В этом случае разрешается ситуация, когда вызываемый метод БЛ прерывает существующую транзакцию, в которую он вовлечён согласно своим настройкам, а вызывающий метод явно об этом не узнаёт.

Создание вложенной транзакции влечёт выброс исключения, если внешняя транзакция уже прервана

1	<code>with sbis.CreateTransaction(...) as outer:</code>
2	<code>outer.Break()</code>
3	<code>try:</code>
4	<code>with sbis.CreateTransaction(...) as inner:</code>
5	<code># Сюда не попадаем, т.к. внешняя</code>
6	<code>транзакция уже прервана.</code>
7	<code>pass</code>
8	<code>except:</code>
9	<code># Попадаем сюда.</code>
	<code>pass</code>

Этот сценарий направлен на оптимизацию ресурсов БЛ: зачем тратить ресурс исполнителя, если результаты работы всё равно будут отброшены.

SQL-запросы в прерванной транзакции влекут выброс исключения

1	<code>with sbis.CreateTransaction(...) as txn:</code>
2	<code>txn.Break()</code>
3	<code>try:</code>
4	<code>sbis.SqlQuery(...)</code>
5	<code>except:</code>
6	<code># Попадаем сюда в любом случае, т.к.</code>
7	<code>транзакция прервана.</code>
	<code>pass</code>

Аналогично предыдущему пункту, этот сценарий направлен на оптимизацию ресурсов.

Игнорирование результата завершения транзакции

1	<code>try:</code>
2	<code>with sbis.CreateTransaction(...,</code>
3	<code>ignore_outcome=True) as outer:</code>

4	<code>with sbis.CreateTransaction(...) as inner:</code>
5	<code>inner.Break()</code>
6	<code>except:</code>
7	<code># Сюда не попадаем, т.к., хотя вложенная</code>
8	<code>транзакция прервана, мы запросили игнорирование</code>
	<code>результата завершения</code>
	<code># текущей (outer) транзакции.</code>
	<code>pass</code>

Этот сценарий применяется, когда результат завершения транзакции не влияет на успешность исполнения метода БЛ. В C++ эта механика реализована с помощью метода явного завершения транзакции [sbis::dbi::Transaction::EndNoThrow\(\)](#).

Метод End() возвращает статус завершения транзакции

В C++ метод явного завершения транзакции [sbis::dbi::Transaction::End\(\)](#) возвращает [статус операции](#):

- `sbis::dbi::TransactionStatus::Committed` - транзакция зафиксирована.
- `sbis::dbi::TransactionStatus::CommitPending` - внешняя транзакция помечена на фиксацию (если `End()` вызван у вложенной транзакции).
- `sbis::dbi::TransactionStatus::RolledBack` - откат транзакции.
- `sbis::dbi::TransactionStatus::RollbackPending` - внешняя транзакция помечена на откат (если `End()` вызван у вложенной транзакции).